Reservation Tables

- The reservation table is a **2D chart**. It shows the **utilisation or reservation of consecutive pipeline stages to evaluate** a specific function across successive pipeline cycles. So, **for every function, there is a single reservation table**.
- A reservation table consists of **rows corresponding to pipeline stages** and **columns to clock time units**.
- The **evaluation time for a function is equal to the total number of clock units** in the reservation table.

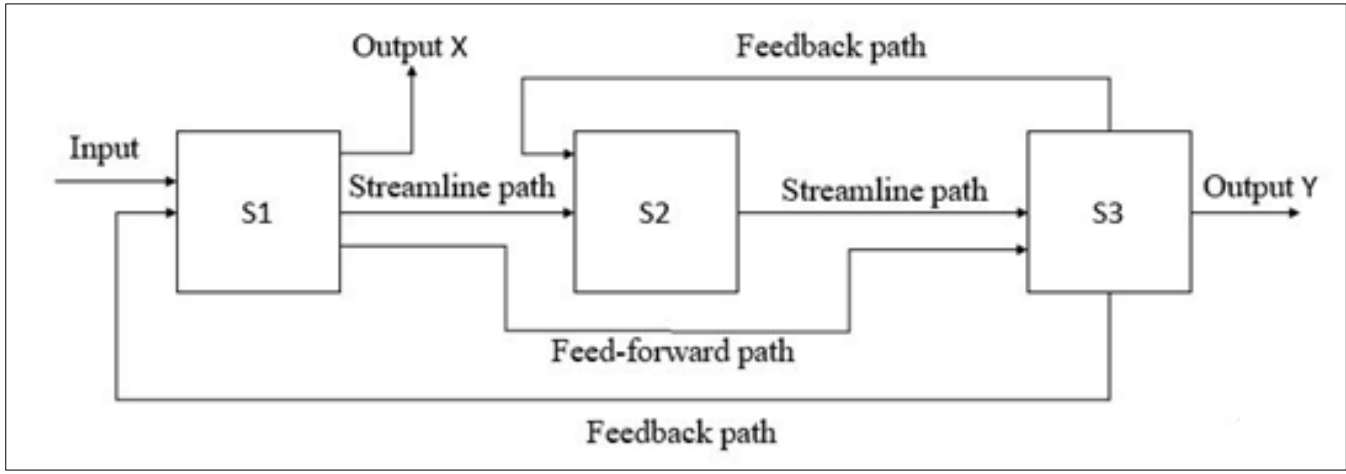
S1	X			
S2		X		
S3			X	
S4				X
Clk	1	2	3	4

Fig: Reservation table for linear pipeline

- In the figure, there is a reservation table for a linear pipeline, where stage-1 (S1) is busy at 1st clock cycle, stage-2 (S2) is busy at 2nd clock cycle, and so on.
- The table represents the flow of data through the pipeline for one complete evaluation of a given function.
- In a static, unifunction pipeline, one reservation table is sufficient to describe the data flow pattern, and one symbol 'X' is sufficient to mark a busy stage.
- In a multifunction pipeline, different reservation tables are used for different functions, and multiple symbols are used to mark different functions, like 'X' and 'Y'.

Features of Reservation Tables

- **Multiple marks in a row:** Indicates **repeated usage of a given stage** if the marks are in distinct columns.
- **Multiple marks in a column:** Indicates that **multiple stages are simultaneously used**.
- **Continuous marks in a row:** If marks are continuous, they may represent **a slower stage needing more time slots**.
- **Number of columns in a table:** It stands for the total time required to execute the program.



- Suppose for a non-linear, multifunctional pipeline, there are two different functions X and Y are executing, and both required different order and different numbers of pipeline stages, such that:

X : S1, S2, S3, S1, S3, S3, S1

S1	X			X			X
S2		X					
S3			X		X	X	
Clk	1	2	3	4	5	6	7

Fig: Reservation table for function X

Y : S1, S3, S2, S3, S1, S2, S3

S1	Y				Y		
S2			Y			Y	
S3		Y		Y			Y
Clk	1	2	3	4	5	6	7

Fig: Reservation table for function Y

Latency Analysis Terminology

❖ **Initiation:** An **initiation** refers to the **start of a single function evaluation.**

❖ **Latency:**

- The number of clock cycles between two initiations of a pipeline is the latency between them.
- Latency values must be positive integers.
- A latency of ‘k’ means, two initiations are separated by k clock cycles.

S1	X					X		X
S2		X		X				
S3			X		X		X	
Clk	1	2	3	4	5	6	7	8

❖ **Collision:**

- An attempt by two or more initiations to use the same pipeline stage at the same time will cause a collision.
- Collision means two or more functions are using the same resource at the same time.
- Collisions must be avoided.
- Some latencies will cause collisions and some will not.

Collision Example

S1	X_1		X_2		X_3	X_1	X_4	X_1, X_2		X_2, X_3	
S2		X_1		X_1, X_2		X_2, X_3		X_3, X_4		X_4	.....
S3			X_1		X_1, X_2		X_1, X_2, X_3		X_2, X_3, X_4		
Clk	1	2	3	4	5	6	7	8	9	10	11

Collision with scheduling latency 2

S1	X					X		X
S2		X		X				
S3			X		X		X	
Clk	1	2	3	4	5	6	7	8

Reservation table for function X

So, Latencies 2 & 5 causes collision

S1	X_1					X_1, X_2		X_1			
S2		X_1		X_1			X_2		X_2		....
S3			X_1		X_1		X_1	X_2		X_2	
Clk	1	2	3	4	5	6	7	8	9	10	11

Collision with scheduling latency 5

Latency Analysis Terminology (cont.)

❖ Forbidden Latencies:

- Latencies that cause collisions are called forbidden latencies.
- To detect a forbidden latency, one need to check the distance between any 2 checkmarks in the same row of the reservation table.
- Consider the reservation table:

S1: {(6-1),(8-1),(8-6)}, i.e. {5,7,2}
S2: {(4-2)} i.e. {2}
S3: {(5-3),(7-3),(7-5)}, i.e. {2,4,2}

S1	X					X		X
S2		X		X				
S3			X		X		X	
Clk	1	2	3	4	5	6	7	8

Taking union of all the forbidden latencies:

Forbidden Latencies = {2,4,5,7}

❖ Permissible Latencies:

- Latencies that does not cause collisions are permissible latencies.
- Permissible latencies from the reservation table

= clock cycles of the pipeline – clock cycles in forbidden latencies

Permissible latencies = $\{1,3,6,8^+\}$ (8^+ means all the latencies after 8 are permissible, so those will not create any collision)

Latency Analysis Terminology (cont.)

❖ **Latency Sequence:**

- A latency sequence is a sequence of permissible latencies between successive task initiations.
- Example: Latency Sequence = Permissible latencies = $\{1,3,6, 8^{+}\}$

❖ **Latency Cycle:**

- A latency sequence that repeats the same subsequence (cycle) indefinitely.

S1	X1					X1	X2	X1				X2	X3	X2				X3	X4	X3	
S2		X1		X1				X2		X2				X3		X3				X4	.....
S3			X1		X1		X1		X2		X2		X2		X3		X3		X3		
Clk	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	

Cycle Cycle Cycle .....

Latency Cycle (6)
= 6,6,6, ...

Cycle Repeats

S1	X1	X2				X1	X2	X1	X2	X3	X4				X3	X4	X3	X4	X5	X6
S2		X1	X2	X1	X2				X1		X3	X4	X3	X4						X5
S3			X1	X2	X1		X1	X2				X3	X4	X3	X4	X3	X4			
Clk	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20

Latency Cycle (1,8)
= 1,8,1,8, ...

Latency Analysis Terminology (cont.)

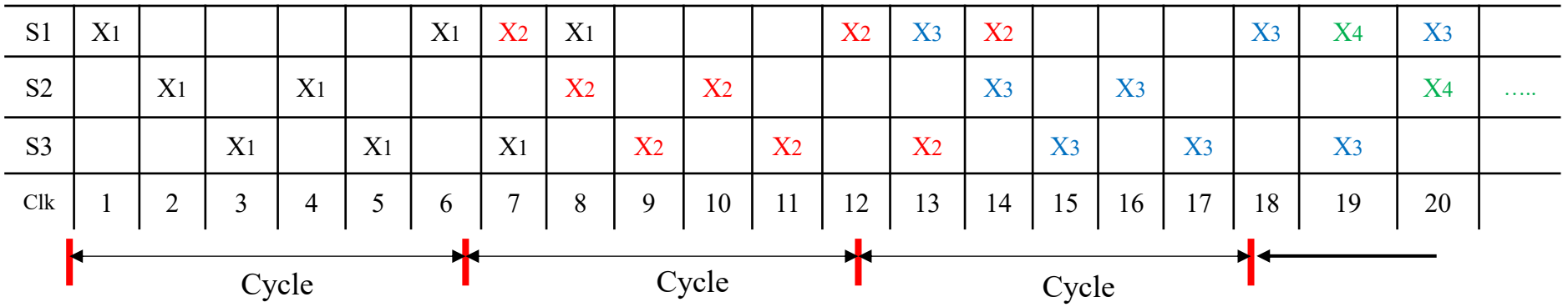
❖ **Average Latency:**

- The average latency of a cycle is the sum of all latencies divided by the number of latencies along the cycle.
- Example: The latency cycle (1,8) has an average latency of (1+8)/2 = 4.5

  The latency cycle (6) has an average latency of 6/1 = 6

❖ **Constant cycle:**

- A constant cycle is a latency cycle that contains only one latency value.
- Example: Latency cycle (6) is a constant cycle = 6, 6, 6, 6,……



Collision Free Scheduling

- To avoid collisions and to achieve maximum throughput, all the tasks that are waiting in the ready queue for initiation, should be scheduled properly.
- When scheduling tasks in a non-linear pipeline, the main objective is to obtain the shortest average latency between initiations without causing collisions.
- Following are the steps to obtain this:

1. **Collision vector calculation** (Determines from forbidden and permissible latencies)
2. **State transition diagram**
3. **Simple cycles and Greedy cycles**
4. **MAL (Minimum Average Latency)**

1. Collision Vector

- The collision vector is a method of analyzing how often we can initiate a new operation into the pipeline and maintain synchronous flow without collision (means permissible state).
- It is a m-bit binary vector C =(C_m C_{m-1} ….. C_3 C_2 C_1) that can be obtained from the combined set of forbidden latencies and permissible latencies. Value of m is the maximum value of forbidden latency. So for the next example m will be 7
- C_i = 1 if latency i causes a collision, i.e. for forbidden latencies

  C_i = 0 for permissible latencies

For Example:

Forbidden Latencies = {2,4,5,7}

Permissible latencies = {1,3,6, 8^+}

S1	X					X		X
S2		X		X				
S3			X		X		X	
Clk	1	2	3	4	5	6	7	8

So, Collision vector will be:

C_7 C_6 C_5 C_4 C_3 C_2 C_1 [Latency positions]

C = (1 0 1 1 0 1 0) = (1011010)

[MSB of the collision vector will be the max value of forbidden latency]

2. State Transition Diagram

- A state diagram is constructed from collision vector.
- A state diagram will specify the permissible state transitions among successive initiations based on the collision vector.
- State diagram shows the all possible initial sequences (cycles) without causing a collision in the pipeline.
- State transition will be the permissible latencies of the initial collision vector.

Example: **Initial Collision Vector (ICV)** = (1011010)

- **Method to draw state transition diagram:**
  - The permissible latencies = $\{1,3,6,8^+\}$ (Permissible latencies are the positions of 0s in the collision vector), so we have to find the next states of collision vector for the transitions $(1,3,6,8^+)$
  - Permissible latencies are the transitions that are used to generate the new states.
  - To find the collision vector of next state, the collision vector of present state is shifted right for each transition $(1,3,6,8^+)$.
  - The shifted vector of **Present State (PS)** is then bitwise OR with **Initial Collision Vector (ICV),** which is initial state of the state transition diagram.
  - We can again generate some new states from the next state, based on the permissible latencies of the next state. This process will continue until we do not get any new state.

2. State Transition Diagram

➢ **Step 1:** Calculating next state with **Present State (PS)** = 1011010

For the 1st step, present state and initial collision vector will be same.

- Present state = Initial state = 1011010
- Permissible latencies = {1,3,6,8+}

▪ **With latency 1:**

```
  1 0 1 1 0 1 0   ICV
+ 0 1 0 1 1 0 1   After right shift of PS by 1
-----------------------------------------------
  1 1 1 1 1 1 1   Next state
```

- So, the **new state 1111111** with latency 1

▪ **With latency 3:**

```
  1 0 1 1 0 1 0   ICV
+ 0 0 0 1 0 1 1   After right shift of PS by 3
-----------------------------------------------
  1 0 1 1 0 1 1   Next state
```

- So, the **new state 1011011** with latency 3

▪ **With latency 6:**

```
  1 0 1 1 0 1 0   ICV
+ 0 0 0 0 0 0 1   After right shift of PS by 6
-----------------------------------------------
  1 0 1 1 0 1 1   Next state
```

- So, same **next state 1011011** with latency 3 and 6

▪ **With latency 8+:**

```
  1 0 1 1 0 1 0   ICV
+ 0 0 0 0 0 0 0   After right shift of PS by 8
-----------------------------------------------
  1 0 1 1 0 1 0   Next state
```

- So, the **next state 1011010** with latency 8, which returns to the initial state.

2. State Transition Diagram

- With latency 1, the next state will be 1111111
- With latency 3 & 6, the next state will be 1011011
- With latency 8^+ the next state will be 1011010

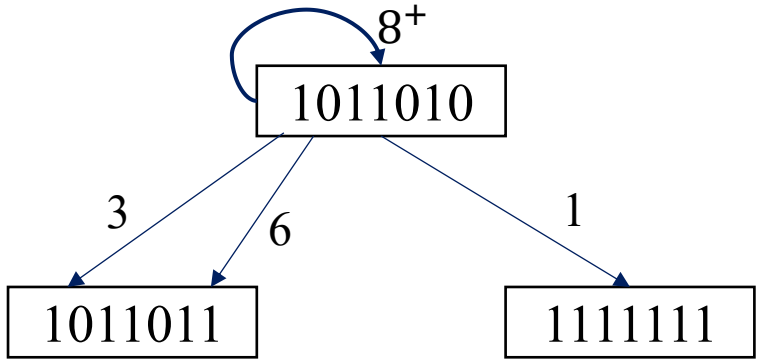


Fig: Initial State Diagram

- Now again, we have to find the transitions of the two new states so that we can go to other new states: 1011011 and 1111111
- Find the permissible latencies of the new states, i.e. the position of 0s in the new states.
- For the new state 1011011, the permissible latencies will be : $\{3,6, 8^+\}$
- For the new state 1111111, the permissible latencies will be : $\{8^+\}$

2. State Transition Diagram

➢ **Step 2:** Calculating next state with **Present State (PS)** = 1011011

- Initial state = 1011010
- Permissible latencies will be : {3,6, 8^+}

▪ **With latency 3:**

1 0 1 1 0 1 0 ICV
\+ 0 0 0 1 0 1 1 After right shift of PS by 3

---

1 0 1 1 0 1 1 **Next state**

- So, the **next state 1011011** with latency 3 is the same present state.

▪ **With latency 6:**

1 0 1 1 0 1 0 ICV
\+ 0 0 0 0 0 0 1 After right shift of PS by 6

---

1 0 1 1 0 1 1 **Next state**

- So, the **next state 1011011** with latency 6 is the same present state.

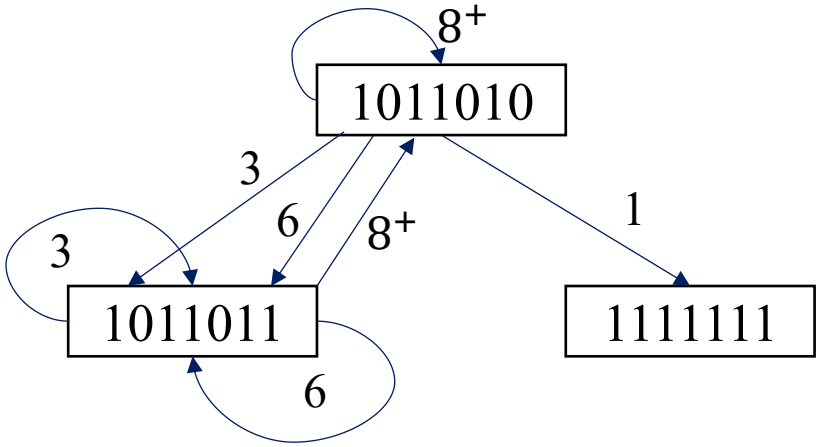
▪ **With latency 8^+:**

1 0 1 1 0 1 0 ICV
\+ 0 0 0 0 0 0 0 After right shift of PS by 8

---

1 0 1 1 0 1 0 **Next state**

- So, the **next state 1011010** with latency 8, which returns to the initial state.



❑ So, no new state generated from the present state 1011011

2. State Transition Diagram

➢ **Step 3:** Calculating next state with **Present State (PS)** = 1111111

- Initial state = 1011010
- Permissible latencies will be : $\{8^+\}$

▪ **With latency 8^+:**

```
  1 0 1 1 0 1 0   ICV
+ 0 0 0 0 0 0 0   After right shift of PS by 8
-----------------------------------------------
  1 0 1 1 0 1 0   Next state
```

- So, the **next state 1011010** with latency 8, which returns to the initial state.
- No new state generated from the present state 1011011

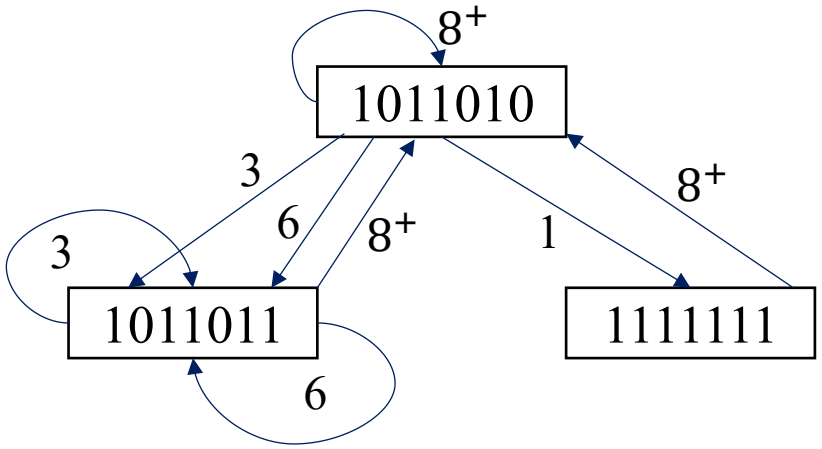


Fig: Final State Diagram

❑ Note: The bitwise OR of the shifted version of the present state with the initial collision vector is meant **to prevent collision from future initiations.**

❑ When the number of shifts is greater than the maximum forbidden latency of the initial collision vector, all transitions are always redirected back to the initial state. For example: in the state diagram, after 8 or more shifts, the next 8^+ state must be the initial state, regardless of which state the transition starts from.

3. Simple and Greedy Cycles

- Cycles can be find out from the state transition diagram.
- **Simple Cycles:** A simple cycle is a latency cycle in which each state appears only once.
- **Greedy Cycles:** A greedy cycle is a simple cycle where the average latency is minimum.

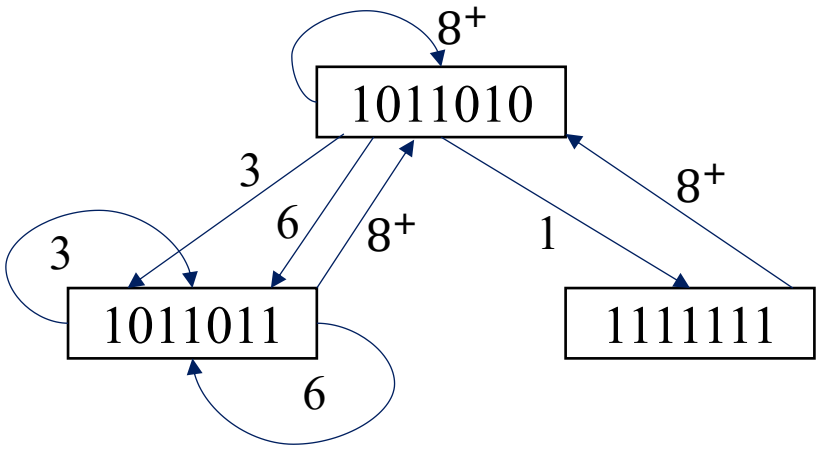


Fig: Final State Diagram

Latency Cycles	Cycle types	Average Latency
(3)	Simple and Constant cycle	3
(6)	Simple and constant cycle	6
(8)	Simple and constant cycle	8
(1,8)	Simple cycle	4.5
(3,8)	Simple cycle	5.5
(6,8)	Simple cycle	7

- As the avg latency of (3) is minimum, so, **Greedy Cycle = (3)**

4. MAL (Minimum Average Latency)

- MAL is the minimum average latency obtained from the greedy cycle.
- Atleast one of the greedy cycles will lead to the MAL

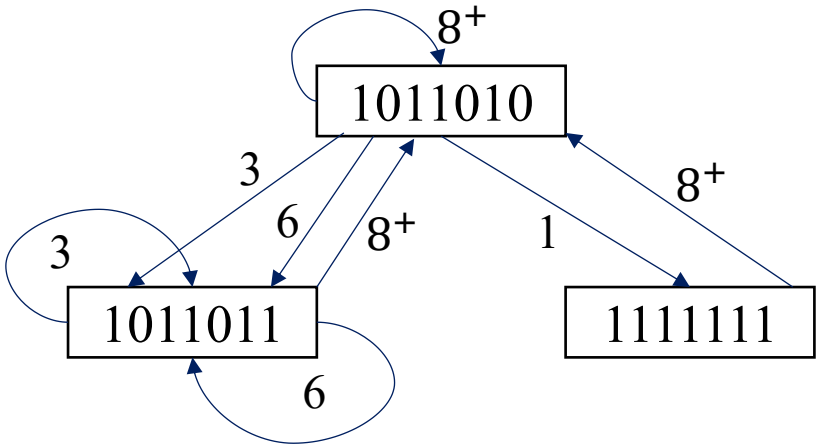
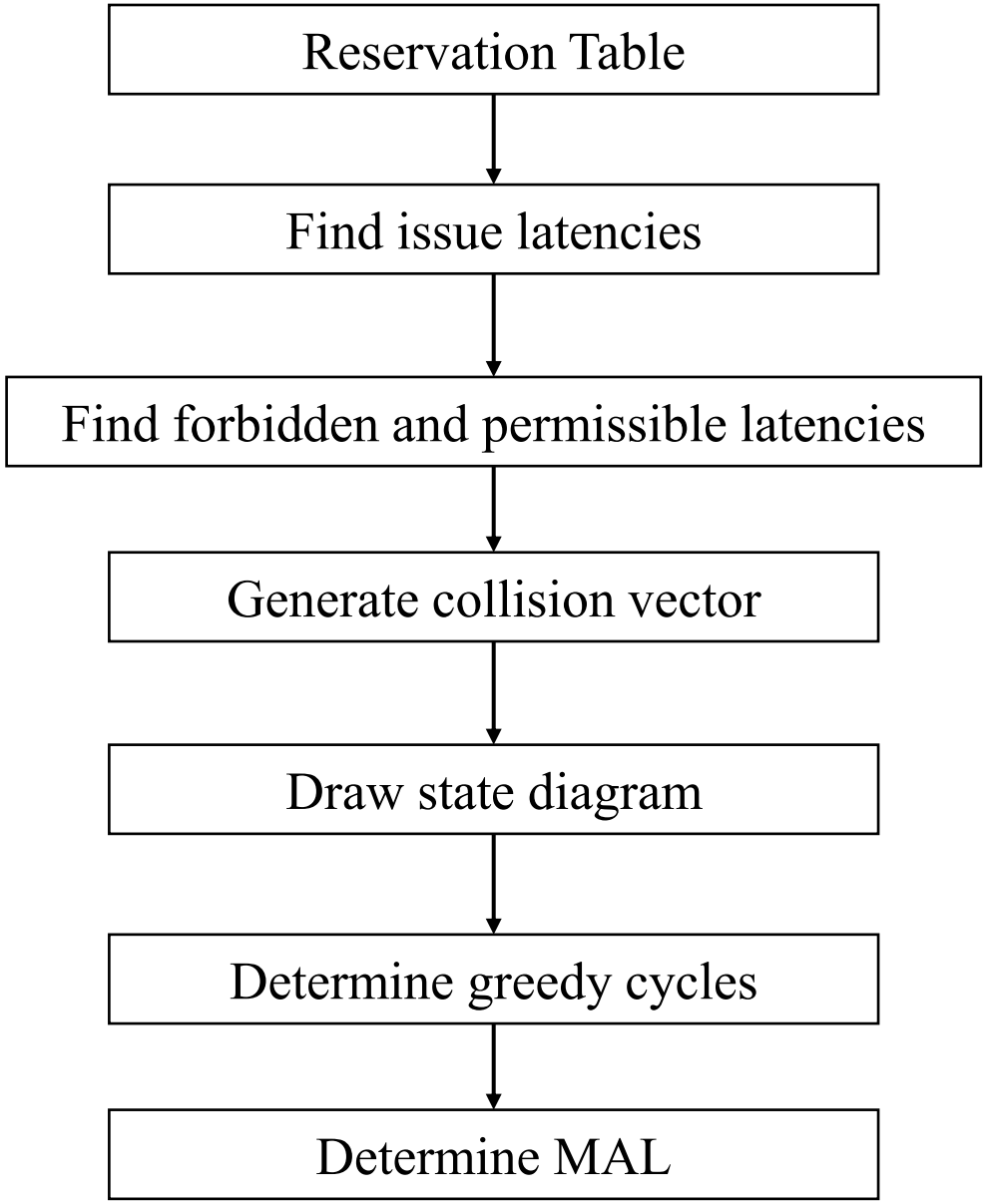


Fig: Final State Diagram

Latency Cycles	Cycle types	Average Latency
(3)	**Simple (Greedy Cycle)**	**3 (MAL)**
(6)	Simple	6
(8)	Simple	8
(1,8)	Simple	4.5
(3,8)	Simple cycle	5.5
(6,8)	Simple cycle	7

- The greedy cycle (3) leads to **MAL = 3**
- As the avg latencies of (3) and (1,8) are minimum, so, Greedy Cycles = **(3) and (1,8)**

Overall Summery to Find The Collision Free Scheduling



Numerical Problem Based on Reservation Table

- *Consider the following reservation table:*

Clk	1	2	3	4	5	6	7
S1	X					X	
S2			X				X
S3		X		X			
S4			X		X		

a) Determine latencies in forbidden list F, and the collision vector C

b) Draw the state transition diagram

c) List all simple and greedy cycles

d) Find optimal constant latency cycle. / What will be the minimum allowed constant cycle?

e) Determine MAL

f) If clock period, T = 20ns, determine the maximum throughput of the pipeline.

Solution

a) Forbidden latencies in each stage:

S1 = {5}, S2 = {4}, S3 = {2}, S4 = {2}

Forbidden latencies = {2, 4, 5}, Permissible latencies = {1,3,6$^+$}

Collision Vector = (1 1 0 1 0)

Clk	1	2	3	4	5	6	7
S1	X					X	
S2			X				X
S3		X		X			
S4			X		X		

b) Step 1 : PS = Initial Collision Vector = 11010, Permissible latencies = {1,3,6$^+$}

- **With latency 1:**

  1 1 0 1 0 ICV
+ 0 1 1 0 1 After right shift of PS by 1

---

  1 1 1 1 1 **Next state**

- So, the **new state 11111** with latency 1

- **With latency 3:**

  1 1 0 1 0 ICV
+ 0 0 0 1 1 After right shift of PS by 3

---

  1 1 0 1 1 **Next state**

- So, the **new state 11011** with latency 3

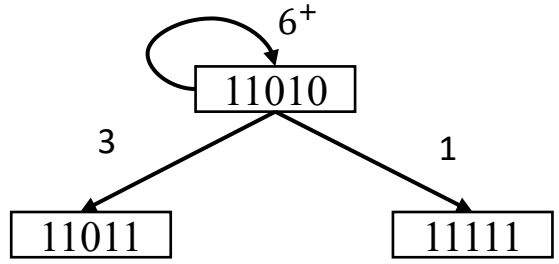
- **With latency 6$^+$:**

  1 1 0 1 0 ICV
+ 0 0 0 0 0 After right shift of PS by 6

---

  1 1 0 1 0 **Next state**

- So, the **next state 11010** with latency 6, which returns to the initial state.



Initial State Diagram

Solution (continue)

b) Step 2: PS =11011 Initial Collision Vector = 11010, Permissible latencies = {3,6+}

- **With latency 3:**

  1 1 0 1 0 ICV
+ 0 0 0 1 1 After right shift of PS by 3
---
  1 1 0 1 1 **Next state**

- So, the **next state 11011** with latency 3 is the same present state.

- **With latency 6+:**

  1 1 0 1 0 ICV
+ 0 0 0 0 0 After right shift of PS by 6
---
  1 1 0 1 0 **Next state**

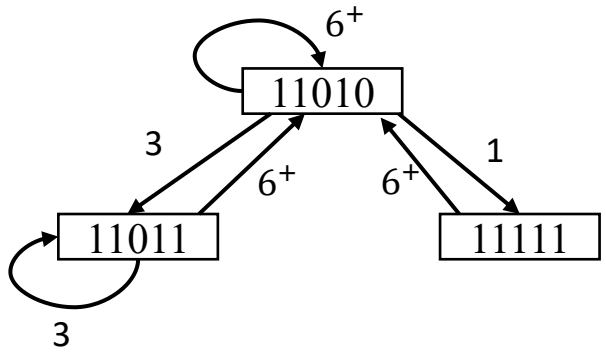
- So, the **next state 11010** with latency 6, which returns to the initial state.

Step 3: PS =11111 Initial Collision Vector = 11010, Permissible latencies = {6+}

- **With latency 6+:**

  1 1 0 1 0 ICV
+ 0 0 0 0 0 After right shift of PS by 6
---
  1 1 0 1 0 **Next state**

- So, the **next state 11010** with latency 6, which returns to the initial state.



Final State Diagram

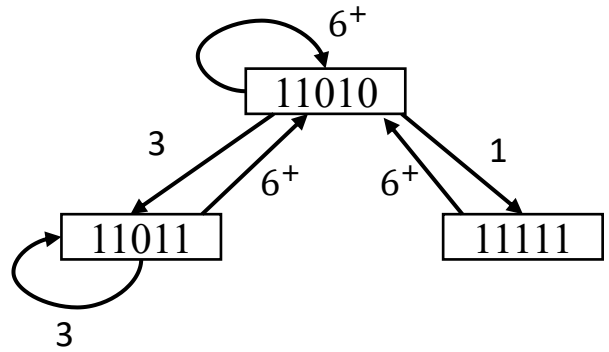
Solution (continue)

c) Simple Cycles : (3), (6), (3,6), (1,6)

Greedy Cycles : (3), (1,6)

d) Optimal Constant Latency Cycle: (3)

e) MAL (Minimum Average Latency) = 3



f) Throughput,

$$W = (\eta / \tau) \times (1 / MAL)$$

Assuming efficiency = 1

$$\tau = 20ns$$

$$\text{Throughput (W)} = \frac{1}{20\times10^{-9}} \times \frac{1}{3} = \frac{10^6}{20\times3\times10^{-3}} = 16.67 \times 10^6 \text{ instructions / sec}$$

$$= 16.67 \text{ MIPS}$$

- Upper bound of MAL = Number of 1's in ICV
- Lower bound of MAL = Maximum no of checkmarks in any row of the reservation table

Numerical Problem (2) Based on Reservation Table

- *Consider the following reservation table:*

Clk	1	2	3	4	5	6
S1	X					X
S2		X			X	
S3			X			
S4				X		
S5		X				X

a) Determine latencies in forbidden list F, and the collision vector C

b) Draw the state transition diagram

c) List all simple and greedy cycles

d) Find optimal constant latency cycle. / What will be the minimum allowed constant cycle?

e) Determine MAL

f) If clock period, T = 10ns, determine the maximum throughput of the pipeline.

Solution

a) Forbidden latencies in each stage:

S1 = {5}, S2 = {3}, S3 = {0}, S4 = {0}, S5 = {4}

Forbidden latencies = {3, 4, 5}, Permissible latencies = {1,2,6$^+$}

Collision Vector = (1 1 1 0 0)

Clk	1	2	3	4	5	6
S1	X					X
S2		X			X	
S3			X			
S4				X		
S5		X				X

b) Step 1 : PS = Initial Collision Vector = 11100, Permissible latencies = {1,2,6$^+$}

- **With latency 1:**

  1 1 1 0 0 ICV
  + 0 1 1 1 0 After right shift of PS by 1

  ---

  1 1 1 1 0 **Next state**

- So, the **new state 11110** with latency 1

- **With latency 2:**

  1 1 1 0 0 ICV
  + 0 0 1 1 1 After right shift of PS by 2

  ---

  1 1 1 1 1 **Next state**

- So, the **new state 11111** with latency 2

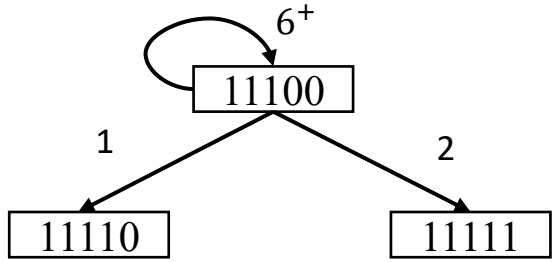
- **With latency 6$^+$:**

  1 1 1 0 0 ICV
  + 0 0 0 0 0 After right shift of PS by 6

  ---

  1 1 1 0 0 **Next state**

- So, the **next state 11100** with latency 6, which returns to the initial state.



Initial State Diagram

Solution (continue)

b) Step 2: PS =11110 Initial Collision Vector = 11100, Permissible latencies = {1,6^+}

- **With latency 1:**

  1 1 1 0 0 ICV
+ 0 1 1 1 1 After right shift of PS by 1
---
  1 1 1 1 1 **Next state**

- So, the **next state 11111** with latency 1 is a previous state.

- **With latency 6^+:**

  1 1 1 0 0 ICV
+ 0 0 0 0 0 After right shift of PS by 6
---
  1 1 1 0 0 **Next state**

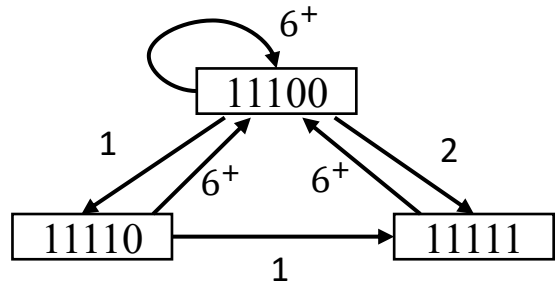
- So, the **next state 11100** with latency 6, which returns to the initial state.

Step 3: PS =11111 Initial Collision Vector = 11100, Permissible latencies = {6^+}

- **With latency 6^+:**

  1 1 1 0 0 ICV
+ 0 0 0 0 0 After right shift of PS by 6
---
  1 1 1 0 0 **Next state**

- So, the **next state 11100** with latency 6, which returns to the initial state.



Final State Diagram

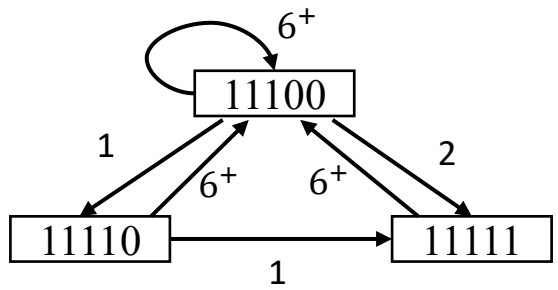
Solution (continue)

c) Simple Cycles : (6), (1,6) ,(2,6), (1,1,6)

Greedy Cycles : (1,6), (1,1,6)

d) Optimal Constant Latency Cycle: (6) [As 6 is the only constant latency cycle]

e) MAL (Minimum Average Latency) = 2.67



f) Throughput,

$$W = (\eta / \tau) \times (1 / MAL)$$

Assuming efficiency = 1

$$\tau = 10ns$$

$$\text{Throughput (W)} = \frac{1}{10\times10^{-9}} \times \frac{1}{2.67} = \frac{10^6}{10\times2.67\times10^{-3}} = 37.45 \times 10^6 \text{ instructions / sec}$$

$$= 37.45 \text{ MIPS}$$

Question on Speedup of Pipeline Architecture

❖ Prove that a k-stage pipeline can be at most k-times faster than that of a non pipelined architecture.

Solution : Let **tp = one clock cycle time for pipelined** architecture.

Time taken for n processes having k segments in pipeline architecture = (k+n-1)*tp

Let **tn = time taken for one process to complete in a non pipeline** architecture.

Time taken for n processes to complete in a non pipeline architecture = n*tn

Speed up ratio for non pipeline and pipeline configuration is = n*tn / (k+n-1)*tp

If n is very large compared to k, i.e. number of processes is very large than number of stages,

then, **(k+n-1) ≈ n** and **(Sk) = tn / tp** [neglecting the number of stages]

If a process takes same time in both case with pipeline and non pipeline configuration then,

tn = k*tp

Thus speed up ratio will, **Sk =k*tp/tp =k**

Theoretically **maximum speedup ratio will be k** where k is the total number of segments in which process is divided in pipelined architecture.

But the maximum speedup cannot be achieved due to some limitations.

Limitations to Achieve Maximum Speedup

- **Different stages of pipeline take different time to complete** their sub-operations, and in pipelining **clock cycle must be chosen equal to maximum time delay among all the stages.** Thus all **other stages have to waste time** waiting for next clock cycle.
- Pipelining can be of maximum benefit if whole process can be divided into **stages which are independent to each other**. But if there is **some resource conflict or data dependency** i.e., a **instruction depends on the result of pervious instruction** which is not yet available then the **instruction has to wait till result generates**.